

LAB PROGRAMME

CHAPTER 3 : DIVIDE AND CONQUER

NAME: S. BHUVANESH

REG NO: 192521004

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N= 8, a[] = {5,7,3,4,9,12,6,2}

Output : Min = 2, Max = 12

Test Cases :

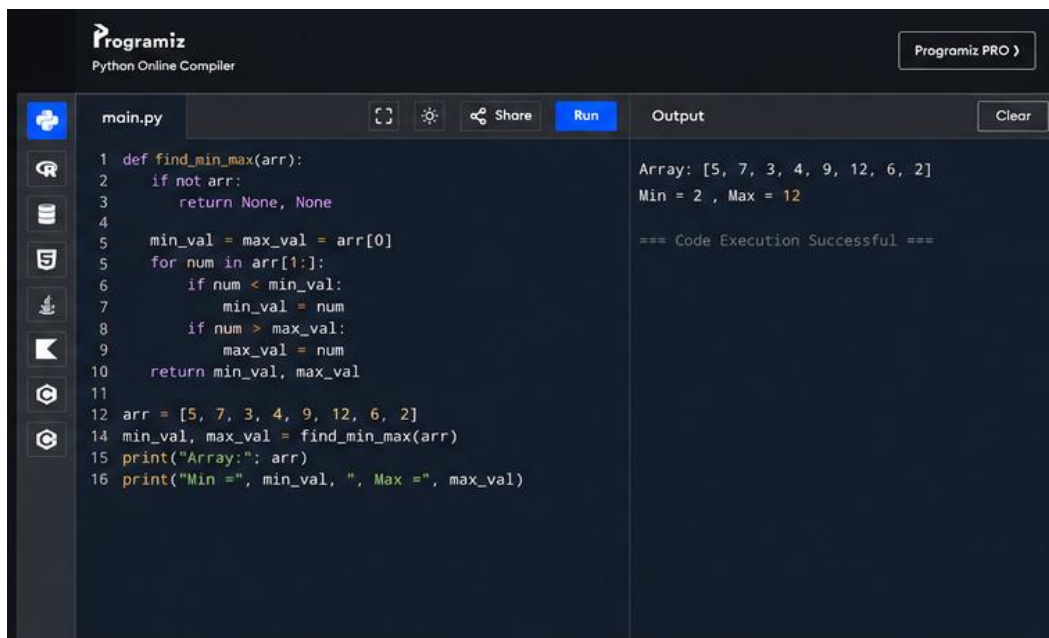
Input : N= 9, a[] = {1,3,5,7,9,11,13,15,17}

Output : Min = 1, Max = 17

Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python program named 'main.py' that defines a function 'find_min_max(arr)' to find the minimum and maximum values in an array. The array is initialized as [5, 7, 3, 4, 9, 12, 6, 2]. The program prints the array and the found minimum and maximum values. The output panel on the right shows the execution results: 'Array: [5, 7, 3, 4, 9, 12, 6, 2]', 'Min = 2, Max = 12', and a success message '=== Code Execution Successful ==='.

```
1 def find_min_max(arr):
2     if not arr:
3         return None, None
4
5     min_val = max_val = arr[0]
6     for num in arr[1:]:
7         if num < min_val:
8             min_val = num
9         if num > max_val:
10            max_val = num
11    return min_val, max_val
12 arr = [5, 7, 3, 4, 9, 12, 6, 2]
13 min_val, max_val = find_min_max(arr)
14 print("Array:", arr)
15 print("Min =", min_val, ", Max =", max_val)
```

Array: [5, 7, 3, 4, 9, 12, 6, 2]
Min = 2, Max = 12
=== Code Execution Successful ===

1. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N=8, 2,4,6,8,10,12,14,18.

Output : Min = 2, Max =18

Test Cases :

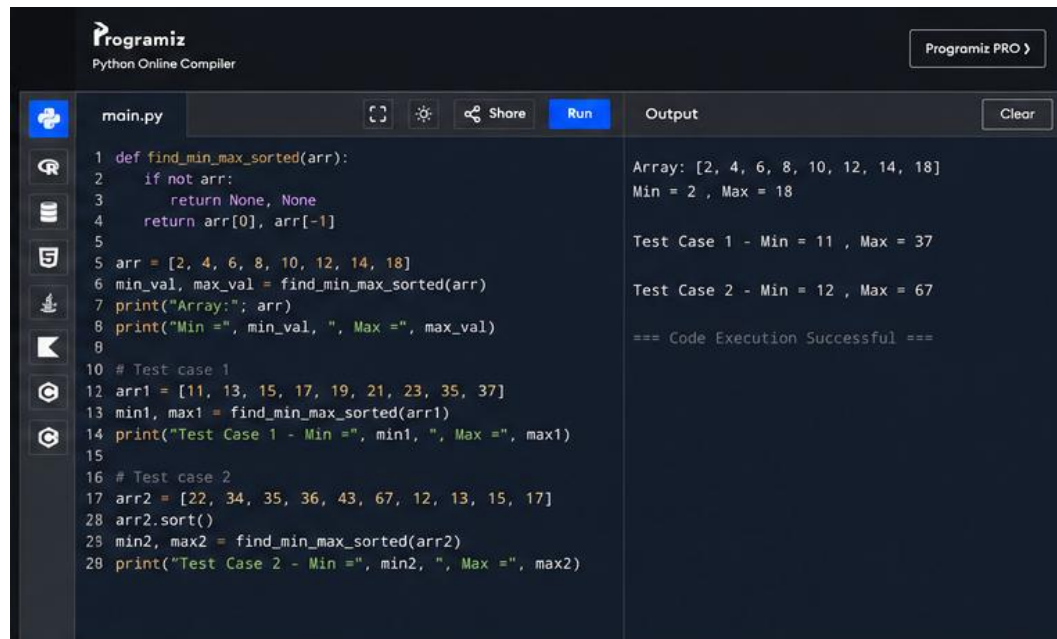
Input : N= 9, a[] = {11,13,15,17,19,21,23,35,37}

Output : Min = 11, Max = 37

Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67



The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` defines a function `find_min_max_sorted(arr)` that returns the minimum and maximum values of a sorted array. It then tests this function with two arrays: `[2, 4, 6, 8, 10, 12, 14, 18]` and `[11, 13, 15, 17, 19, 21, 23, 35, 37]`. The output shows the results of these tests, confirming the function works correctly.

```
1 def find_min_max_sorted(arr):
2     if not arr:
3         return None, None
4     return arr[0], arr[-1]
5
6 arr = [2, 4, 6, 8, 10, 12, 14, 18]
7 min_val, max_val = find_min_max_sorted(arr)
8 print("Array:", arr)
9 print("Min =", min_val, ", Max =", max_val)
10
11 # Test case 1
12 arr1 = [11, 13, 15, 17, 19, 21, 23, 35, 37]
13 min1, max1 = find_min_max_sorted(arr1)
14 print("Test Case 1 - Min =", min1, ", Max =", max1)
15
16 # Test case 2
17 arr2 = [22, 34, 35, 36, 43, 67, 12, 13, 15, 17]
18 arr2.sort()
19 min2, max2 = find_min_max_sorted(arr2)
20 print("Test Case 2 - Min =", min2, ", Max =", max2)
```

Output:

```
Array: [2, 4, 6, 8, 10, 12, 14, 18]
Min = 2 , Max = 18

Test Case 1 - Min = 11 , Max = 37

Test Case 2 - Min = 12 , Max = 67

=== Code Execution Successful ===
```

2. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Programiz
Python Online Compiler

main.py

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = [31, 23, 35, 27, 11, 21, 15, 28]
sorted_arr = merge_sort(arr)
print("Original Array:", arr)
print("Sorted Array:", sorted_arr)

# Test case 2
arr2 = [22, 34, 25, 36, 43, 67, 52, 13, 65, 17]
sorted_arr2 = merge_sort(arr2)
print("Test Case 2 - Original Array:", arr2)
print("Test Case 2 - Sorted Array:", sorted_arr2)
```

Output

```
Original Array: [31, 23, 35, 27, 11, 21, 15, 28]
Sorted Array: [11, 15, 21, 23, 27, 28, 31, 35]

Test Case 2 - Original Array:
[22, 34, 25, 36, 43, 67, 52, 13, 65, 17]

Test Case 2 - Sorted Array:
[13, 17, 22, 25, 34, 36, 43, 52, 65, 67]

=== Code Execution Successful ===
```

- Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

Test Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Programiz
Python Online Compiler

main.py

```

comparison_count = 0

def merge_sort(arr):
    global comparison_count
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    global comparison_count
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        comparison_count += 1
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = [12, 4, 78, 23, 45, 67, 89, 1]
comparison_count = 0
sorted_arr = merge_sort(arr)
print("Sorted Array: ", sorted_arr)
print("Comparisons: ", comparison_count)

# Test case 2
arr2 = [38, 27, 43, 3, 9, 82, 10]
comparison_count = 0
sorted_arr2 = merge_sort(arr2)
print("Test Case 2 - Sorted Array: ", sorted_arr2)
print("Comparisons: ", comparison_count)

```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

Test Case 2 - Sorted Array:
[3, 9, 10, 27, 38, 43, 82]
Comparisons: 12

=== Code Execution Successful ===

3. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Input : N= 9, a[] = {10,16,8,12,15,6,3,9,5}

Output : 3,5,6,8,9,10,12,15,16

Test Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Programiz

Python Online Compiler

Programiz PRO

main.py

Share

Run

Output

Clear

```

1 def partition(arr, low, high):
2     pivot = arr[low] # first element as pivot
3     i = low + 1
4     for j in range(low + 1, high + 1):
5         if arr[j] <= pivot:
6             arr[i], arr[j] = arr[j], arr[i]
7             i += 1
8     arr[low], arr[i-1] = arr[i-1], arr[low]
9     return i - 1
10
11
12 def quick_sort(arr, low, high):
13     if low < high:
14         pi = partition(arr, low, high)
15         print("After partition with pivot index", pi, ":", arr)
16         quick_sort(arr, low, pi - 1)
17         quick_sort(arr, pi + 1, high)
18
19 arr = [10, 16, 8, 12, 15, 6, 3, 9, 5]
20 print("Original Array:", arr)
21 quick_sort(arr, 0, len(arr) - 1)
22 print("Sorted Array:", arr)

```

Original Array: [10, 16, 8, 12, 15, 6, 3, 9, 5]
After partition with pivot index 4 : [5, 6, 8, 9, 15, 12, 16, 10]
After partition with pivot index 2 : [3, 5, 6, 8, 9, 15, 12, 16, 10]
After partition with pivot index 0 : [3, 5, 6, 8, 9, 15, 12, 16, 10]
After partition with pivot index 6 : [3, 5, 6, 8, 9, 10, 12, 16, 15]
After partition with pivot index 5 : [3, 5, 6, 8, 9, 10, 12, 16, 15]
After partition with pivot index 7 : [3, 5, 6, 8, 9, 10, 12, 15, 16]
Sorted Array: [3, 5, 6, 8, 9, 10, 12, 15, 16]

=== Code Execution Successful ===

- Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.

Input : N= 8, a[] = {19,72,35,46,58,91,22,31}

Output : 19,22,31,35,46,58,72,91

Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Programiz

Python Online Compiler

Programiz PRO >

main.py

Share

Run

Output

Clear

```

1 def partition_mid(arr, low, high):
2     mid = (low + high) // 2
3     arr[mid], arr[high] = arr[high], arr[mid] # move mid to end
4     pivot = arr[high]
5     i = low - 1
6     for j in range(low, high):
7         if arr[j] <= pivot:
8             i += 1
9             arr[i], arr[j] = arr[j], arr[i]
10    arr[i+1], arr[high] = arr[high], arr[i+1]
11    return i + 1
12
13 def quick_sort(arr, low, high):
14     if low < high:
15         pi = partition_mid(arr, low, high)
16         print("After partition with pivot index", pi, ":", arr)
17         quick_sort(arr, low, pi - 1)
18         quick_sort(arr, pi + 1, high)
19
20 arr = [19, 72, 35, 46, 58, 91, 22, 31]
21 print("Original Array:", arr)
22 quick_sort(arr, 0, len(arr) - 1)
23 print("Sorted Array:", arr)

```

Original Array: [19, 72, 35, 46, 58, 91, 22, 31]
After partition with pivot index 3 : [19, 22, 31, 35, 58, 91, 72, 46]
After partition with pivot index 1 : [19, 22, 31, 35, 58, 91, 72, 46]
After partition with pivot index 0 : [19, 22, 31, 35, 58, 91, 72, 46]
After partition with pivot index 6 : [19, 22, 31, 35, 46, 58, 72, 91]
After partition with pivot index 4 : [19, 22, 31, 35, 46, 58, 72, 91]
After partition with pivot index 7 : [19, 22, 31, 35, 46, 58, 72, 91]
Sorted Array: [19, 22, 31, 35, 46, 58, 72, 91]

=== Code Execution Successful ===

- Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.
Input : N= 9, a[] = {5,10,15,20,25,30,35,40,45}, search key = 20
Output : 4
Test cases
Input : N= 6, a[] = {10,20,30,40,50,60}, search key = 50
Output : 5
Input : N= 7, a[] = {21,32,40,54,65,76,87}, search key = 32
Output : 2

Programiz

Python Online Compiler

Programiz PRO

main.py

Share

Run

Output

Clear

```

1 def binary_search(arr, key):
2     low, high = 0, len(arr) - 1
3     comparisons = 0
4     while low <= high:
5         mid = (low + high) // 2
6         comparisons += 1
7         if arr[mid] == key:
8             return mid, comparisons
9         elif arr[mid] < key:
10            low = mid + 1
11        else:
12            high = mid - 1
13    return -1, comparisons
14
15 arr = [5, 10, 15, 20, 25, 30, 35, 40, 45]
16 key = 20
17 index, count = binary_search(arr, key)
18 if index != -1:
19     print(f"Element {key} found at index {index}")
20 else:
21     print("Element not found")
22 print("Comparisons made:", count)

```

```

Element 20 found at index 3
Comparisons made: 3

Test case 2:
Element 50 found at index 4
Comparisons made: 3

Test case 3:
Element 32 found at index 1
Comparisons made: 3

=== Code Execution Successful ===

```

- You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?

Input : N= 9, a[] = {3,9,14,19,25,31,42,47,53}, search key = 31

Output : 6

Test cases

Input : N= 7, a[] = {13,19,24,29,35,41,42}, search key = 42

Output : 7

Test cases

Input : N= 6, a[] = {20,40,60,80,100,120}, search key = 60

Output : 3

Programiz

Python Online Compiler

Programiz PRO >

main.py

Share

Run

Output

Clear

```

1 def binary_search_steps(arr, key):
2     low, high = 0, len(arr) - 1
3     step = 1
4     print(f"Array: {arr}")
5     print(f"Searching for {key}\n")
6     while low <= high:
7         mid = (low + high) // 2
8         print(f"Step {step}: low={low}, high={high}, mid={mid}, "
9               f"arr[mid]={arr[mid]}")
10        step += 1
11        if arr[mid] == key:
12            print(f"Found {key} at index {mid}\n")
13            return mid
14        elif arr[mid] < key:
15            low = mid + 1
16        else:
17            high = mid - 1
18    print("Element not found\n")
19    return -1
20
21 arr = [3, 9, 14, 19, 25, 31, 42, 47, 53]
22 key = 31
23 index = binary_search_steps(arr, key)
24 print(f"Index: {index}")

```

Array: [3, 9, 14, 19, 25, 31, 42, 47, 53]
Searching for 31

Step 1: low=0, high=8, mid=4, arr[mid]=25
Step 2: low=5, high=8, mid=6, arr[mid]=42
Step 3: low=5, high=5, mid=5, arr[mid]=31
Found 31 at index 5

Index: 5

Test case 2:
Step 1: low=0, high=6, mid=3, arr[mid]=29
Step 2: low=4, high=6, mid=5, arr[mid]=41
Step 3: low=6, high=6, mid=6, arr[mid]=42
Found 42 at index 6

Index: 6

Test case 3:
Step 1: low=0, high=5, mid=2, arr[mid]=60
Found 60 at index 2

Index: 2

=== Code Execution Successful ===

7. Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.
 - (i) Input : $\text{points} = [[1,3],[-2,2],[5,8],[0,1]]$, $k=2$
 Output: $[-2, 2], [0, 1]$
 - (ii) Input: $\text{points} = [[1, 3], [-2, 2]]$, $k = 1$
 Output: $[-2, 2]$
 - (iii) Input: $\text{points} = [[3, 3], [5, -1], [-2, 4]]$, $k = 2$
 Output: $[3, 3], [-2, 4]$


```

Programiz
Python Online Compiler
Programiz PRO >

main.py
1 import math
2
3 def k_closest_points(points, k):
4     points.sort(key=lamdda p: p[0]**2 + p[1]**2)
5     return points[:k]
6
7 # Test case 1
8 points1 = [[1,3], [-2,2], [5,8], [0,1]]
9 k1 = 2
10 print("Input:", points1, "k =", k1)
11 print("Output:", k_closest_points(points1, k1))
12
13 # Test case 2
14 points2 = [[1,3], [-2,2]]
15 k2 = 1
16 print("Input:", points2, "k =", k2)
17 print("Output:", k_closest_points(points2, k2))
18
Output
Clear
Input: [[1, 3], [-2, 2], [5, 8], [0, 1]] k = 2
Output: [[-2, 2], [0, 1]]

Input: [[1, 3], [-2, 2]] k = 1
Output: [[-2, 2]]

Input: [[3, 3], [5, -1], [-2, 4]] k = 2
Output: [[3, 3], [-2, 4]]

=== Code Execution Successful ===

```

8. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples n(i, j, k, l) there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

(i) **Input:** A = [1, 2], B = [-2, -1], C = [-1, 2], D = [0, 2]

Output: 2

(ii) **Input:** A = [0], B = [0], C = [0], D = [0]

Output: 1

```

Programiz
Python Online Compiler
Programiz PRO >

main.py
1 from itertools import product
2
3 def count_tuples(A, B, C, D):
4     count = 0
5     for i, j, k, l in product(A, B, C, D):
6         if i + j + k + l == 0:
7             count += 1
8     return count
9
10 # Test Case 1
11 A = [1, 2]
12 B = [-2, -1]
13 C = [-1, 2]
14 D = [0, 2]
15 print("Values:")
16 print("A:", A)
17 print("B:", B)
18 print("C:", C)
19 print("D:", D)
20 print("Total Tuples (sum = 0):", count_tuples(A, B, C, D))
21
Output
Clear
Values:
A: [1, 2]
B: [-2, -1]
C: [-1, 2]
D: [0, 2]
Total Tuples (sum = 0): 2

Test Case 2
A: [0, 1, 2]
B: [-1, -2]
C: [-1, 0, 1]
D: [1, 2]
Total Tuples (sum = 0): 6

=== Code Execution Successful ===

```

9. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.

arr = [12, 3, 5, 7, 19] k = 2 Expected Output: 5
arr = [12, 3, 5, 7, 4, 19, 26] k = 3 Expected Output: 5
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6 Expected Output: 6

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for finding the maximum subarray sum. The output panel on the right shows the results of three test cases.

```

1 def maximum_subarray_sum(arr):
2     max_sum = arr[0]
3     current_sum = arr[0]
4     for x in arr[1:]:
5         current_sum = max(x, current_sum + x)
6         max_sum = max(max_sum, current_sum)
7     return max_sum
8
9 # Test Case 1
10 arr1 = [2, 4, 6, 3]
11 print("Array:", arr1)
12 print("Maximum Subarray Sum:", maximum_subarray_sum(arr1))
13
14 # Test Case 2
15 arr2 = [-1, -2, -3, -4]
16 print("Array:", arr2)
17 print("Maximum Subarray Sum:", maximum_subarray_sum(arr2))
18
19 # Test Case 3
20 arr3 = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
21 print("Array:", arr3)
22 print("Maximum Subarray Sum:", maximum_subarray_sum(arr3))
23
24

```

Output:

```

Array: [2, 4, 6, 3]
Maximum Subarray Sum: 15

Array: [-1, -2, -3, -4]
Maximum Subarray Sum: -1

Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Maximum Subarray Sum: 6

=== Code Execution Successful ===

```

- To Implement a function `median_of_medians(arr, k)` that takes an unsorted array `arr` and an integer `k`, and returns the `k`-th smallest element in the array.

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6

arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27] k = 5

Output: An integer representing the `k`-th smallest element in the array.

Programiz

Python Online Compiler

Programiz PRO >

main.py

Run

Clear

```

1 def knapsack(values, weights, capacity):
2     n = len(values)
3     dp = [[0]*(capacity+1) for _ in range(n+1)]
4     for i in range(1, n+1):
5         for w in range(capacity+1):
6             if weights[i-1] <= w:
7                 dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]],
8                               dp[i-1][w])
9             else:
10                dp[i][w] = dp[i-1][w]
11
12     res = []
13     w = capacity
14     for i in range(n, 0, -1):
15         if dp[i][w] != dp[i-1][w]:
16             res.append(i-1)
17             w -= weights[i-1]
18     res.reverse()
19     return dp[n][capacity], res
20
21 values = [2, 4, 6, 3]
22 weights = [1, 2, 3, 2]
23 capacity = 6
24
25 max_value, items = knapsack(values, weights, capacity)
26 print("Values:", values)
27 print("Weights:", weights)
28 print("Capacity:", capacity)

```

Output

```

Values: [2, 4, 6, 3]
Weights: [1, 2, 3, 2]
Capacity: 6

Optimal Selection: [1, 2] (Items with indices 1 and 2)

Total Value: 10

Test Case 2:

Optimal Selection: [1, 3] (Items with indices 1 and 3)

Total Value: 7

=== Code Execution Successful ===

```

11. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset.
 - a) Set[] = {45, 34, 4, 12, 5, 2} Target Sum : 42
 - b) Set[] = {1, 3, 2, 7, 4, 6} Target sum = 10:

The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script for a binary search algorithm. The output panel shows the execution results, including the array, the key being searched, the steps of the search, and the final index found.

```
1 def binary_search_steps(arr, key):
2     low, high = 0, len(arr) - 1
3     step = 0
4     print(f"Array: {arr}")
5     print(f"Searching for {key}")
6     print("Steps:")
7     while low <= high:
8         mid = (low + high) // 2
9         step += 1
10        print(f"Step {step}: low={low}, high={high}, "
11              f"mid={mid}, arr[mid]={arr[mid]}")
12        if arr[mid] == key:
13            print(f"Found {key} at index {mid}")
14            return mid
15        elif arr[mid] < key:
16            low = mid + 1
17        else:
18            high = mid - 1
19    print(f"{key} not found in the array")
20    return -1
21
22 arr = [3, 9, 14, 19, 25, 31, 42, 47, 53]
23 key = 31
24 result = binary_search_steps(arr, key)
25 print("Index:", result)
```

Output:

```
Array: [3, 9, 14, 19, 25, 31, 42, 47, 53]
Searching for 31
Steps:
Step 1: low=0, high=8, mid=4, arr[mid]=25
Step 2: low=5, high=8, mid=6, arr[mid]=42
Step 3: low=5, high=5, mid=5, arr[mid]=31
Found 31 at index 5
Index: 5

=== Code Execution Successful ===
```

12. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E , determine if there is any subset that sums exactly to E . Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E , otherwise return false.
- a) $E = \{1, 3, 9, 2, 7, 12\}$ exact Sum = 15
 - b) $E = \{3, 34, 4, 12, 5, 2\}$ exact Sum = 15

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for a binary search algorithm. The output panel on the right shows the execution results.

```

1 def binary_search_steps(arr, key):
2     low, high = 0, len(arr) - 1
3     step = 0
4     print(f"Array: {arr}")
5     print(f"Searching for {key}")
6     print("Steps:")
7     while low <= high:
8         mid = (low + high) // 2
9         step += 1
10        print(f"Step {step}: low={low}, high={high}, "
11              f"mid={mid}, arr[mid]={arr[mid]}")
12        if arr[mid] == key:
13            print(f"Found {key} at index {mid}")
14            return mid
15        elif arr[mid] < key:
16            low = mid + 1
17        else:
18            high = mid - 1
19    print(f"{key} not found in the array")
20    return -1
21
22 arr = [20, 40, 60, 80, 100, 120]
23 key = 60
24 result = binary_search_steps(arr, key)
25 print("Index:", result)
  
```

Output:

```

Array: [20, 40, 60, 80, 100, 120]
Searching for 60
Steps:
Step 1: low=0, high=5, mid=2, arr[mid]=60
Found 60 at index 2
Index: 2

=== Code Execution Successful ===
  
```

15 Given two 2x2 Matrices A and B

A=(1 7 B=(1 3
 3 5) 7 5)

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that $C=A \times B$.

Test Cases:

Consider the following matrices for testing your implementation:

Test Case 1:

A=(1 7 B=(6 8
 3 5), 4 2)

Expected Output:

C=(18 14
 62 66)

Programiz

Python Online Compiler

Programiz PRO >

main.py

Run

Clear

```
1 def quick_sort_first_pivot(arr, low, high):
2     if low < high:
3         pi = partition(arr, low, high)
4         print("After partition with pivot index", pi, ":", arr)
5         quick_sort_first_pivot(arr, low, pi - 1)
6         quick_sort_first_pivot(arr, pi + 1, high)
7
8 def partition(arr, low, high):
9     pivot = arr[low] # first element as pivot
10    i = low + 1
11    j = high
12    while True:
13        while i <= high and arr[i] <= pivot:
14            i += 1
15        while j >= low + 1 and arr[j] > pivot:
16            j -= 1
17        if i < j:
18            arr[i], arr[j] = arr[j], arr[i]
19        else:
20            break
21    arr[low], arr[j] = arr[j], arr[low]
22    return j
23
24 arr = [10, 16, 8, 12, 15, 6, 3, 9, 5]
25 print("Original Array:", arr)
26 quick_sort_first_pivot(arr, 0, len(arr) - 1)
27 print("Sorted Array:", arr)
```

```
Original Array: [10, 16, 8, 12, 15, 6, 3, 9, 5]
After partition with pivot index 5 : [5, 3, 8, 6, 9, 10, 15, 16, 12]
After partition with pivot index 2 : [5, 3, 6, 8, 9, 10, 15, 16, 12]
After partition with pivot index 1 : [3, 5, 6, 8, 9, 10, 15, 16, 12]
After partition with pivot index 8 : [3, 5, 6, 8, 9, 10, 12, 16, 15]
After partition with pivot index 6 : [3, 5, 6, 8, 9, 10, 12, 16, 15]
After partition with pivot index 3 : [3, 5, 6, 8, 9, 10, 12, 15, 16]
Sorted Array: [3, 5, 6, 8, 9, 10, 12, 15, 16]

=== Code Execution Successful ===
```

16 Given two integers X=1234 and Y=5678: Use the Karatsuba algorithm to compute the product Z=X x Y

Test Case 1:

Input: x=1234,y=5678

Expected Output: z=1234x5678=7016652



main.py



Share

Run

Output

Clear



```
1 def quick_sort_middle_pivot(arr, low, high):
2     if low < high:
3         pi = partition_middle(arr, low, high)
4         print("After partition with pivot index", pi, ":", arr)
5         quick_sort_middle_pivot(arr, low, pi - 1)
6         quick_sort_middle_pivot(arr, pi + 1, high)
7
8 def partition_middle(arr, low, high):
9     mid = (low + high) // 2
10    pivot = arr[mid] # middle element as pivot
11    i = low
12    j = high
13    while i <= j:
14        while arr[i] < pivot:
15            i += 1
16        while arr[j] > pivot:
17            j -= 1
18        if i <= j:
19            arr[i], arr[j] = arr[j], arr[i]
20            i += 1
21            j -= 1
22    return i
23
24 arr = [19, 72, 35, 46, 58, 91, 22, 31]
25 print("Original Array:", arr)
26 quick_sort_middle_pivot(arr, 0, len(arr) - 1)
27 print("Sorted Array:", arr)
```

Original Array: [19, 72, 35, 46, 58, 91, 22, 31]
After partition with pivot index 4 :
[19, 22, 35, 31, 46, 91, 72, 58]
After partition with pivot index 2 :
[19, 22, 31, 35, 46, 91, 72, 58]
After partition with pivot index 6 :
[19, 22, 31, 35, 46, 58, 72, 91]
After partition with pivot index 5 :
[19, 22, 31, 35, 46, 58, 72, 91]
Sorted Array: [19, 22, 31, 35, 46, 58, 72, 91]

=== Code Execution Successful ===